

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО

ОТЧЁТ
ПО ДИСЦИПЛИНЕ «БЭК-ЭНД РАЗРАБОТКА»
по теме:
ЛР2: РЕАЛИЗАЦИЯ МИКРОСЕРВИСОВ (СЕРВИС АРЕНДЫ
НЕДВИЖИМОСТИ)

Проверил:

подпись, дата

Добряков Д. И.

Выполнил:

подпись, дата

Субботин Александр

1 Задача

Опираясь на документ ДЗ4, реализовать разделение монолитного бэкенд-приложения (ЛР1) на микросервисы.

Стек: Go 1.23, Gin, PostgreSQL 16, JWT, Docker Compose.

2 Ход работы

2.1 1. Структура проекта

Монолит разделён на 4 микросервиса и API Gateway. Каждый сервис — отдельный Go-модуль со своим `go.mod`, `Dockerfile` и миграциями.

```
lab2/
├── api-gateway/           # порт 8080 – маршрутизация
├── user-service/          # порт 8081 – auth, users
├── property-service/      # порт 8082 – properties, favorites
├── booking-service/       # порт 8083 – bookings, reviews
├── chat-service/          # порт 8084 – chats, messages
└── docker-compose.yaml
```

Внутренняя структура каждого сервиса:

```
├── src/service/main.go
├── internal/
│   ├── config/config.go
│   ├── handler/
│   ├── middleware/auth.go
│   ├── model/
│   └── repository/
├── migrations/001_init.sql
├── Dockerfile
├── go.mod
└── go.sum
```

2.2 2. Разделение базы данных

Используется единый экземпляр PostgreSQL с 4 логически изолированными базами (`database-per-service`). Каждый сервис подключается к своей базе:

Сервис	БД
user-service	user_db
property-service	property_db
booking-service	booking_db
chat-service	chat_db

При первом запуске PostgreSQL выполняет init-скрипт `01_create_databases.sh`, который создаёт все 4 базы и применяет миграции.

FK между таблицами разных сервисов заменены на UUID-ссылки без constraint. Например, `bookings.property_id` и `bookings.tenant_id` хранят UUID, но не имеют FK-ограничения, т.к. таблицы находятся в разных БД.

2.3 3. Реализация сервисов

2.3.1 3.1 user-service

Отвечает за регистрацию, аутентификацию и профиль пользователя.

Эндпоинты:

- `POST /api/v1/auth/register` — регистрация
- `POST /api/v1/auth/login` — выдача JWT-токена
- `GET /api/v1/users/me` — профиль текущего пользователя

Ключевые решения:

- JWT выдаётся с полем `user_id` и сроком 24 часа.
- Пароли хэшируются через `bcrypt`.
- Миграция: таблица `users`.

2.3.2 3.2 property-service

Управляет объявлениями и избранным.

Публичные эндпоинты:

- CRUD для `/api/v1/properties`
- `GET/POST/DELETE /api/v1/favorites/{propertyId}`
- `GET /api/v1/users/me/properties`

Internal API (для booking-service):

- `GET /internal/properties/{propertyId}` — данные объекта (цена, владелец)
- `GET /internal/properties?owner_id=...` — список ID объектов владельца

Миграция: таблицы properties, property_photos, amenities, property_amenities, favorites.

2.3.3 3.3 booking-service

Управляет бронированиями и отзывами.

Эндпоинты:

- POST/GET /api/v1/bookings — создание/список бронирований
- GET/PATCH /api/v1/bookings/{bookingId} — просмотр/обновление статуса
- GET /api/v1/users/me/owner/bookings — бронирования объектов владельца
- POST /api/v1/reviews — создание отзыва

Межсервисное взаимодействие:

- HTTP-клиент (internal/client/property.go) обращается к property-service через internal API.
- При создании бронирования: получает price_per_night для расчёта суммы.
- При проверке прав: получает owner_id для авторизации действий владельца.
- При листинге бронирований владельца: получает список property_ids.

Миграция: таблицы bookings, reviews, payments.

2.3.4 3.4 chat-service

Управляет чатами и сообщениями.

Эндпоинты:

- GET /api/v1/chats — список чатов пользователя
- GET /api/v1/chats/{chatId}/messages — сообщения чата
- POST /api/v1/chats/{chatId}/messages — отправка сообщения

Миграция: таблицы chats, chat_participants, messages.

2.3.5 3.5 api-gateway

Единая точка входа. Реализован как reverse proxy на Gin.

Маршрутизация:

- /auth/*, /users/me → user-service:8081
- /properties/*, /favorites/*, /users/me/properties → property-service:8082
- /bookings/*, /reviews, /users/me/owner/bookings → booking-service:8083
- /chats/* → chat-service:8084

Передаёт все заголовки (включая Authorization) без изменений.

2.4 4. Аутентификация

Каждый сервис содержит собственный middleware/auth.go, который:

1. Извлекает JWT из заголовка Authorization: Bearer <token>.
2. Валидирует подпись (общий JWT_SECRET).
3. Сохраняет user_id в контекст Gin.

2.5 5. Docker Compose

Файл docker-compose.yml описывает 7 контейнеров:

- 1 PostgreSQL (с 4 логическими БД, healthcheck, автоматическое применение миграций)
- 1 RabbitMQ (брокер сообщений)
- 4 микросервиса (зависят от PostgreSQL)
- 1 API Gateway (зависит от всех сервисов)

Все сервисы подключаются к одному PostgreSQL-контейнеру (postgres:5432), указывая нужную БД через переменную DB_NAME. Сетевое взаимодействие обеспечивается стандартной bridge-сетью Docker Compose.

2.6 6. Запуск

```
cd lab2
docker-compose up --build
```

После запуска API доступен на `http://localhost:8080/api/v1/`.

2.7 7. Тестирование и документация API

Для проверки корректности работы реализованных микросервисов использовалась автоматически сгенерированная Swagger-документация. Каждый сервис предоставляет собственный набор REST API эндпоинтов и отдельный интерфейс Swagger UI.

2.7.1 7.1 user-service

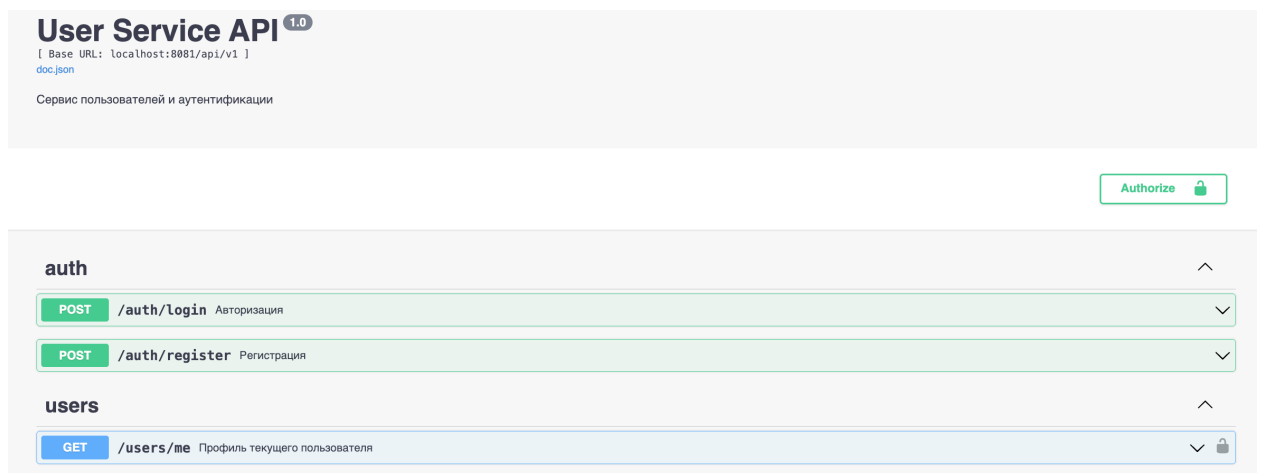


Рисунок 1 — Swagger UI сервиса пользователей (user-service)

На рисунке представлена Swagger-документация сервиса пользователей. Через данный интерфейс выполняется тестирование регистрации пользователей, аутентификации и получения информации о текущем пользователе.

2.7.2 7.2 property-service

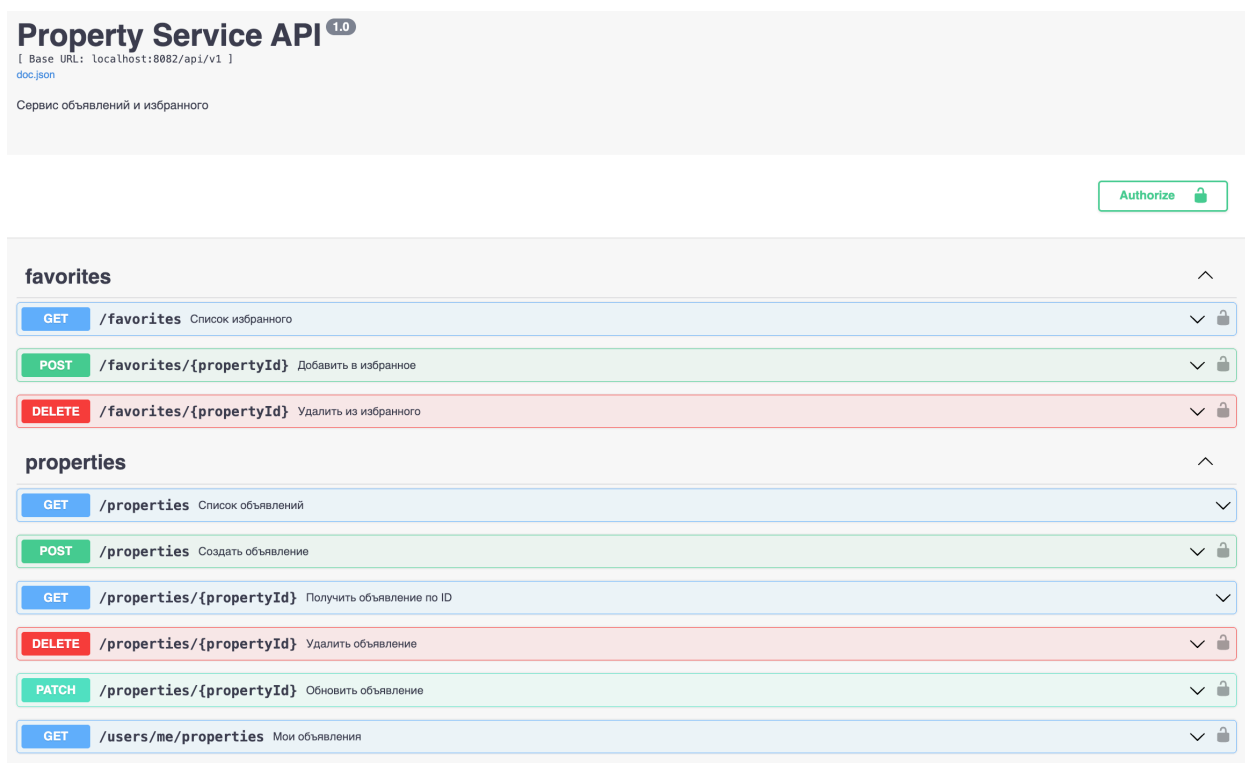


Рисунок 2 — Swagger UI сервиса объектов недвижимости (property-service)

На рисунке представлена документация сервиса недвижимости. Доступны операции создания, редактирования, удаления и поиска объектов недвижимости, а также работа с избранными объектами пользователя.

2.7.3 7.3 booking-service

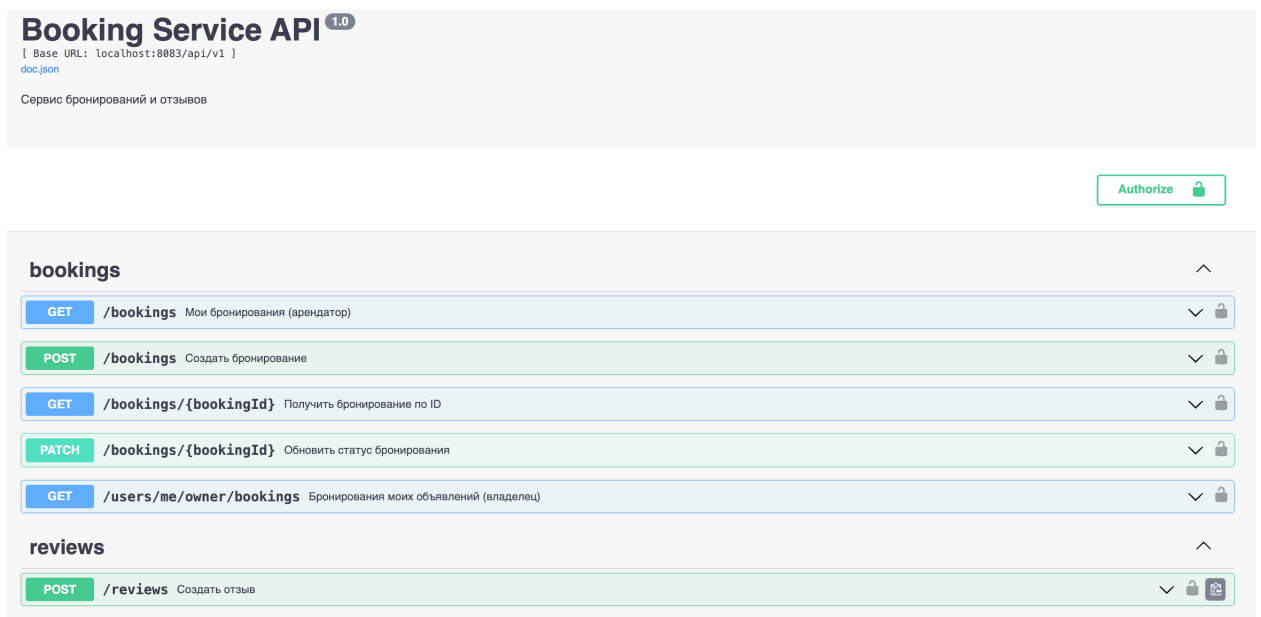


Рисунок 3 — Swagger UI сервиса бронирований (booking-service)

На рисунке показана документация сервиса бронирований. Через API выполняется создание бронирований, изменение их статусов, получение списков бронирований и работа с отзывами.

2.7.4 7.4 chat-service

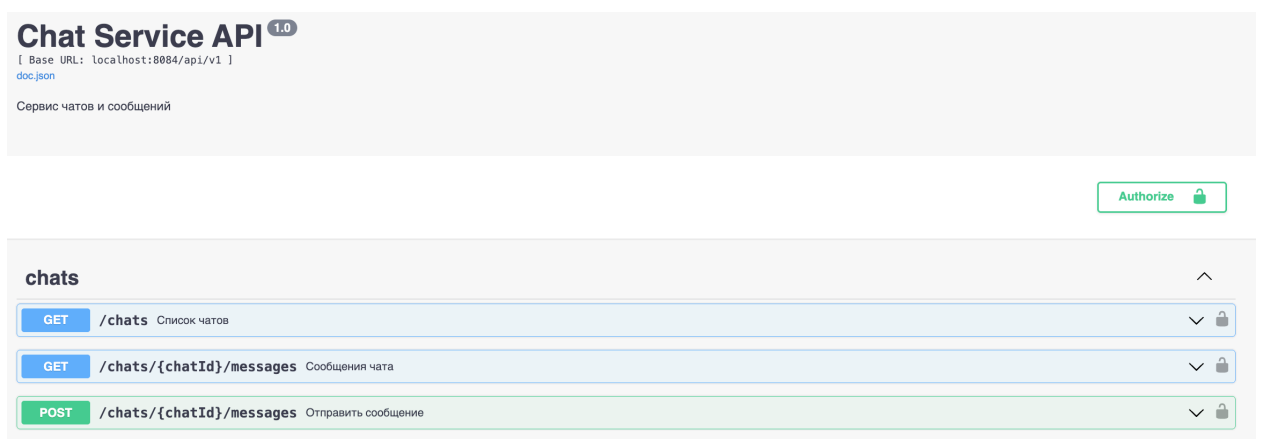


Рисунок 4 — Swagger UI сервиса сообщений (chat-service)

На рисунке представлена документация сервиса обмена сообщениями. Сервис обеспечивает создание и просмотр чатов, а также отправку сообщений между пользователями платформы.

2.7.5 7.5 Internal API property-service



Рисунок 5 — Документация внутреннего API property-service

На рисунке показана документация внутреннего API сервиса недвижимости. Данный интерфейс используется другими микросервисами для получения информации об объектах недвижимости без прямого доступа к базе данных. Такой подход соответствует принципам микросервисной архитектуры и обеспечивает слабую связанность компонентов системы.

3 Вывод

Реализовано разделение монолитного приложения на 4 микросервиса с паттерном database-per-service (4 логических БД в едином PostgreSQL). Каждый сервис имеет собственную БД, Go-модуль и Dockerfile. Межсервисное взаимодействие реализовано через синхронные HTTP-вызовы (internal API). Единая точка входа — API Gateway на порту 8080. Всё развёрнуто через Docker Compose (7 контейнеров).